Module Interface Specification for Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

January 9, 2026

1 Revision History

Date	Version	Notes
Nov 6 - Nov 14	1.0	Preliminary Draft

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at SRS

Contents

3 Introduction

The following document details the Module Interface Specifications for a Physiotherapy Companion app. The application is designed to identify users using computer vision for their prescribed physiotherapy exercise. In addition, an analysis will be performed on the user's movement and, after completion, the app will provide metrics to the user on their performance.

This tool utilizes OpenCV and MediaPipe for image recognition and then performs measurements on the number of repetitions, range of motion and perceived exertion.

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at https://github.com/M9Huynh/technically-functional.

4 Notation

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003). The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | ... | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Physiotherapy Companion.

Data Type	**Notation**	**Description**
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)
float	F	floating point numbers within (approx.) (-$1.798*10^{308}$, $1.798*10^{308}$)
object	X	an abstract representation of a collection of the above, similar to tuple
array	a	a collection of a single data type, n entries of the form (a_0, a_1, a_2, ... ,a_{n-1})
list	L	an array of non-fixed length

The specification of Physiotherapy Companion uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Physiotherapy Companion uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	**Level 2**
Hardware-Hiding Module	
Behaviour-Hiding Module	User Interface Module Home Module Main Module Draw Module
Software Decision Module	Database Management Module Generator Module Metrics Module Analyze Module Feedback Module

Table 1: Module Hierarchy

6 Variables

For clarity and readability, the variables used throughout the document are provided below.

Table 2: Variable used throughout MIS

Name	Type	Description	Used in sections
name	String	Name of the user	7.1
role	String	Values can be PT or patient	7.1
email	String	Used as login ID	7.1
password	String	Set by user for login purposes	7.1
birthday	String	User's birthday for verification	7.1
license_no	Int	Verification of PT	7.1

Table 2: Variable used throughout MIS (continued)

Name	Type	Description	Used in sections
invite_code	String	Given by PT to patient for registration	7.1
date	String	Date selected by user	7.1
time	String	Time selected by user	7.2
recent_analysis	ADT	Most recently generated analysis	7.1
recent_report	Record	Generated report of recent_video	TBD
recent_video	String (JSON)	Last video recorded by patient	TBD
video	String (JSON)	A video in the user database	TBD
report	Record	Report(s) sent to Generator module for analysis OR requested by patient	TBD
ex_instruct	String	Performance instructions of requested exercise	TBD
sets	String	Number of sets given by PT	7.1
repetitions	String	Repetitions of exercise per set	7.1
rest_time	String	Rest time between sets	7.1
leg_landmarks	Array	MediaPipe output of leg	7.4
processed_frame	Image	Video frame with overlaid markers	7.4
PT_comment	String	PT's comment on patient activity	7.3
overall_analysis	Abstract object	the overall analysis of all patient activity	7.4
processed_frames_list	List(Image object)	list of processed_frames	7.3
feedback_instructions	Abstract Object	instructions given from analysis to feedback module	7.3
fixed_metrics	float	corrected metrics	7.3
initial/final_ plane/time/height	float	initial and final measurements	7.3
ins_ui	String	PT's comment on patient activity	TBD
unprocessed_frames	List(Image objects)	Unprocessed frames list	TBD
rt_metrics	List(of float)	Real-time metrics	TBD
input_frame	Image	Input frame	TBD
exercise	String	Selected exercise	TBD
insights	String	Generated insights	TBD
all_reports	List[report]	All reports are stored in a list	TBD
starting_frame	Image	The last frame before recording	TBD

Table 2: Variable used throughout MIS (continued)

Name	Type	Description	Used in sections
good_to_record	String	A message informing user that the system is ready to record	TBD

7 MIS

This section describes individual module specifications.
Note: Some of the mathematics used are preliminary and not all-encompassing. They are defined below for feedback purposes.

7.1 Module - Database Management

ADT

This module creates an instance of UserAccount_P/PT and is an adapter that connects to the user database. This database consists of these tables:

1. User data: this consists of information such as name, role, email, password, birthday, and license_no/invite_code.
2. Exercise data: this table consists of exercise instructions, maximum height (the highest point their leg can lift up to), minimum height (starting point) and user_account_p['Name'].
3. User activity: This will contain information about a specific patient's exercise activity. Data such as exercise duration, maximum height, number of repetitions, target area, the date the exercise was performed and its time, rest time, number of sets and analysis. Additionally, it will include a column for the patient's feedback on their exercise performance.

7.1.1 Uses

- name
- role
- email
- password
- birthday

- license_no
- invite_code
- exercise
- time
- date
- duration
- recent_video
- recent_ analysis
- Sets
- repetitions
- rest_time

7.1.2 Syntax

Exported Constants

- *user_acc_p*: ADT
- *user_acc_pt*: ADT

Exported Access Programs

[1] *For these functions, the output depends on context. For example, if this function was going to be used in another one which corresponds to the PT functionality of viewing a patient, their view would be restricted to patients under their care.*

Access Routine Name	Input	Output	Exceptions
Description: retrieves user account information from the user table			
get_userdb_info[1]	*Name*	*user_acc_p* *user_acc_pt*	UserNotFoundError DownloadError
Description: allows PT to delete a patient account from the system			
PTaccount_delete	*Name* *email*	NA	FieldEmptyError UserNotFoundError
Description: verifies if username and password match for authentication			
username_pw_match	*email* *password*	NA	LoginMatchError
Description: adds analysis results to user activity history			
add_analysis_to _user_act	*user_acc_p* *recent_analysis*	NA	UserNotFoundError SaveError
Description: saves video recording for user account			
save_video	*user_acc_p* *recent_video*	NA	UserNotFoundError UploadError VideoTooShortError
Description: retrieves analysis data for a specific date			
get_analysis	*user_acc_p* *date*	*recent_analysis*	UserNotFoundError NoAnalysisError DownloadError

Table 3: Exported Access Routines (Part 1)

Access Routine Name	Input	Output	Exceptions
Description: provides exercise instructions to user			
exerc_instruct	*user_acc_p* *exercise*	*ex_instruct*	UserNotFoundError DownloadError NoExerciseError
Description: allows physiotherapist to modify exercise parameters			
PT_modifies_exercise	*user_acc_p* *sets* *repetitions* *rest_time*	*ex_instruct*	UserNotFoundError DownloadError NoExerciseError

Table 4: Exported Access Routines (Part 2)

7.1.3 Semantics

State Variables

A: an instance of user_acc_p
B: an instance of user_acc_p
C: an instance of user_acc_pt
$i,\ j,\ k$: index count

State Invariant

$i, j, k \in \mathbb{Z}^+$
$\forall A, B, C \in UserDatabase : A[i] \neq B[j] \neq C[k]$
$0 < A < length(A) \cap A[i] \notin \emptyset$
$0 < B < length(B) \cap B[i] \notin \emptyset$
$0 < C < length(C) \cap C[i] \notin \emptyset$

7.1.4 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired functionality

7.1.5 Assumptions

User data table is arranged alphabetically.

7.1.6 Access Routine Semantics

get_userdb_info():
NA *(There is no transition taking place)*

PTaccount_delete():

1. From the home screen, PT accesses their patient list
2. PT selects the desired patient's name
3. PT is directed to profile of patient
4. PT selects *'Delete Patient'*
5. The selected patient is deleted from the user database

username_pw_match():
NA

Note: the functions are under development and may change over the course of the project

7.1.7 Local Functions

Access Routine Name	**Input**	**Output**	**Exceptions**
Description: Creates a new user account			
account_create	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired
Description: Sets user information in database			
set_user_info	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	NA	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired

Table 5: Account Management Access Routines (Part 1)

Access Routine Name	Input	Output	Exceptions
Description: Retrieves analysis data from database			
send_db_analysis	*UserAccount_P*	Recent_analysis	User_not_found Download_error
Description: Retrieves video data from database			
send_db_video	*UserAccount_P*	Recent_video	User_not_found Download_error
Description: Sets analysis report for user			
set_analysis_report	*UserAccount_P*	Recent_analysis	User_not_found Download_error

Table 6: Account Management Access Routines (Part 2)

7.2 Module - Generator

This module's purpose is to display the most recent report, an overall analysis of progress, and rehabilitation insights.

7.2.1 Syntax

Exported Constants
NA
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Generates health insights after assessing all past reports			
generate_insights	*all_reports*	*insights*	NoActivityError NoConnectionError
Description: Generates a specific report			
generate_report	*date* *time*	*recent_report*	CannotGenerateError NoActivityError
Description: Retrieves specific report from database			
get_report	*Date* *Time*	*Report*	NA

Table 7: Generator Access Routines

7.2.2 Semantics

State Variables

A: an instance of user_acc_p
E: an instance of recent_analysis
V: video
R: report
RR: recent_report

State Invariant

$\forall RR$, $\exists(A \cap V)$
$\forall R$, $\exists(A \cap V_i | i \geq 1..n, n \in 1..\mathbb{N})$

7.2.3 Environment Variables

NA

7.2.4 Assumptions

NA

7.2.5 Access Routine Semantics

TBD

7.3 Module - Account Creation

This module is responsible for creating a new user account.

7.3.1 Uses

- name
- role
- email
- password
- birthday
- acc_id

7.3.2 Syntax

Exported Constants

- email
- password
- name
- birthday [2]
- acc_double

[2] *'Birthday' is an exported constant for distinguishing two users that may have the same name.*

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: This checks if an account already exists in the user database.			
acc_exists	*name* *birthday*	*acc_double*	FieldEmptyError UserNotFoundError

Table 8: Exported Access Routines (Part 1)

7.3.3 Semantics

State Variables

id: acc_id
name: name
role: role
email: email
dob: birthday
pw: password
acc_copy: acc_double
new_id: newly created acc_id

State Invariant

$\neg\ acc\_copy \implies \neg(\exists\ id)$
$\forall(name \cap role \cap email \cap dob \cap pw) \implies \neg\ (new\_id) = \emptyset$

7.3.4 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

7.3.5 Assumptions

- Users with the same name have different birthdays.

7.3.6 Access Routine Semantics

TBD

7.3.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Creates a new user account			
account_create	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	user_account_p or user_account_pt acc_id	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired
Description: Sets user information in database			
set_user_info	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	NA	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired

Table 9: Account Management Access Routines

7.4 Module - Valid user

This module's purpose is to check whether a potential user is authorized to create an account.

7.4.1 Uses

- name
- role
- invite_code
- license

7.4.2 Syntax

Exported Constants
is_valid: boolean

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: This checks if a physiotherapist's license is valid			
license_check	*name* *license*	*is_valid*	FieldEmptyError InvalidCredsError
Description: This checks if a new patient's invite code is valid.			
invite_code_check	*name* *invite_code*	*is_valid*	FieldEmptyError InvalidCredsError

Table 10: Exported Access Routines

State Variables
license: license_no
invite: invite_code

State Invariant
NA?

7.4.3 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

7.4.4 Assumptions

- Invite codes are randomly generated.
- No two users will have the same invite codes.

7.4.5 Access Routine Semantics

TODO

7.4.6 Local Functions

NA?

7.5 Module - User Account

This module's purpose is to handle the users' accounts in the user database.

7.5.1 Uses

- email
- password
- name
- birthday
- acc_id
- role

7.5.2 Syntax

Exported Constants
name: name
id: acc_id

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: retrieves user account information from the user table			
get_userdb_info[2]	*Name*	*user_acc_p* *user_acc_pt*	UserNotFoundError DownloadError
Description: allows PT to delete a patient account from the system			
PTaccount_delete	*Name* *email*	NA	FieldEmptyError UserNotFoundError
Description: verifies if username and password match for authentication			
username_pw_match	*email* *password*	NA	LoginMatchError

Table 11: Exported Access Routines

[2] *For this function, the output depends on context. For example, if this function was going to be used in one which corresponds to the PT functionality of viewing a patient, their view would be restricted to patients under their care.*

State Variables
email: email

pw: password
name: name
id: acc_id
D(x): 'PT_account_delete(x)'
db: 'User Account database'

State Invariant
$\exists\ acc\_id \mid (email \cap pw \notin \emptyset)$
$D(acc\_id) \neq \emptyset \implies acc\_id \notin db$

7.5.3 Environment Variables

screen: The application's display to the user
touchscreen: Users will click on the screen to select their desired action

7.5.4 Assumptions

- Invite codes are randomly generated.
- No two users will have the same invite codes.

7.5.5 Access Routine Semantics

TODO

7.5.6 Local Functions

NA?

7.6 Module - Exercise Data

7.7 Module - User Activity

7.8 Module - Generator

This module's purpose is to display the most recent report, an overall analysis of progress, and rehabilitation insights.

7.8.1 Syntax

Exported Constants
NA
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Generates health insights after assessing all past reports			
generate_insights	*all_reports*	*insights*	NoActivityError NoConnectionError
Description: Generates a specific report			
generate_report	*date* *time*	*recent_report*	CannotGenerateError NoActivityError
Description: Retrieves specific report from database			
get_report	*Date* *Time*	*Report*	NA

Table 12: Generator Access Routines

7.8.2 Semantics

State Variables
A: an instance of user_acc_p
E: an instance of recent_analysis
V: video
R: report
RR: recent_report

State Invariant
$\forall RR, \exists (A \cap V)$
$\forall R, \exists (A \cap V_i | i \geq 1..n, n \in 1..\mathbb{N})$

7.8.3 Environment Variables

NA

7.8.4 Assumptions

NA

7.8.5 Access Routine Semantics

TBD

7.9 Module - Home

This module displays past exercises, allows viewing of user profile, and directs to the Generator module. PTs are able to comment on their patient's submissions.

7.9.1 Syntax

Exported Constants
NA

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Displays user profile information			
view_profile	*Name*	UserAccount_P OR UserAccount_PT	User_not_found Download_error
Description: Displays analysis report for user			
display_report	*Name* *email*	NA	required_field_empty patient_not_found
Description: Displays summary information			
display_summary	*Email* *Password*	NA	wrong_email_password
Description: Shows patient's past exercises			
p_past_exercises	*Email*	NA	no_activity
Description: Provides physical therapist feedback on patient			
PT_feedback_on_P	*PTcomment*	NA	User_not_found Download_error

Table 13: Home Exported Access Routines

7.9.2 Semantics

State Variables
NA
State Invariant
NA

7.9.3 Environment Variables

NA

7.9.4 Assumptions

TBD

7.9.5 Access Routine Semantics

TBD

7.9.6 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets profile from generator			
get_profile_gen	*NA*	UserAccount_P OR UserAccount_PT	Cannot_access_db

Table 14: Account Management Access Routines

7.10 Module - Main

This module handles the video recording, placing markers on it and checking if the patient's form is appropriate for performing the exercise. It does this through the use of external libraries (notably OpenCV).

7.10.1 Uses

7.10.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Gets metrics from metrics module			
get_metrics	*NA*	*angle* *height* *duration*	DataRetrievalError
Description: Sends overall analysis to home module			
overall_analysis_home	*name* *birthday*	*overall_analysis*	DataRetrievalError AnalysisNotFoundError
Description: Sends real time processed frames to metrics (to be forwarded to Analysis)			
analysis_rt_frame	*processed_frame*	*instructions_ui*	CannotAnalyzeError NoInputError
Description: Performs overall analysis (given at end of recording)			
analysis_overall	*recent_video*	recent_analysis	InsufficientDataError ProcessError
Description: Module will provide corrections (received from Metrics) to UI (e.g. angle is too small)			
corrections_metrics	*rt_metrics*	*NA*	User_not_found Download_error
Description: Sends recent analysis to home module			
send_analysis_home	*NA*	recent_analysis	UserNotFoundError

Table 15: Access Routines for Main Module

Access Routine Name	Input	Output	Exceptions
Description: Calls draw to give markers of the leg			
set_markers	unprocessed_frame	processed_frame	InvalidFrameError

Table 16: Access Routines for Main Module (Continued)

7.10.3 Semantics

State Variables

State Invariant

7.10.4 Environment variables

7.10.5 Assumptions

7.10.6 Access Routine Semantics

7.10.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets starting form of user			
get_initial_form	*input_frame*	unprocessed_frame	InvalidFrameError

Table 17: Local Access Routines (Main)

7.11 Module - Analyze

7.11.1 Uses

7.11.2 Syntax

Exported Constants
Exported Access Programs

7.11.3 Semantics

State Variables
TBD
State Invariant
TBD

Access Routine Name	Input	Output	Exceptions
Description: gets metrics from metrics module			
latest_metrics	*rt_metrics*	*NA*	InsufficientDataError
Description: sends overall analysis to Feedback module			
send_analysis_to_feedback	*recent_analysis*	*NA*	AnalysisNotFoundError
Description: performs real time analysis			
analysis_rt	*processed_frames*	recent_analysis	User_not_found Download_error
Description: overall analysis (given at end of recording)			
analysis_overall	*video*	recent_analysis	ConnectionError InsufficientDataError
Description: module will correct form (eg. if angle is too small)			
adjust_metrics	*processed_frame*	rt_metrics	UndetectedFormError
Description: send overall analysis to Database module			
send_analysis_to_db	*overall_analysis*	*NA*	User_not_found Download_error

Table 18: Access Routines for Analysis Module

7.11.4 Environment variables

NA

7.11.5 Assumptions

7.11.6 Access Routine Semantics

7.11.7 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Gets initial landmarks from the starting position			
get_initial_landmarks	*NA*	UserAccount_P OR UserAccount_PT	User_not_found Download_error

Table 19: Access Routines for Analysis Module

7.12 Module - Feedback

This module is responsible for delivering the feedback (e.g. to adjust form) to the Main module.

7.12.1 Uses

7.12.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: feedback to user given after analysis			
ex_feedback_to_main	*feedback*	*NA*	FeedbackEmptyError
Description: instructions given by analysis to construct feedback that will be forwarded to the Main module			
given_instruct	*feedback_instructions*	*NA*	InsEmptyError

Table 20: Access Routines for Analysis Module

7.12.3 Semantics

State Variables
TODO
State Invariant
TODO

7.12.4 Assumptions

TODO

7.12.5 Access Routine Semantics

TODO

7.12.6 Local Functions

7.13 Module - Metrics

This module measures metrics (*angle, duration, height*) that are necessary to construct any corrections to the user's form.

TODO

Access Routine Name	Input	Output	Exceptions
Description: Constructs feedback based on instructions from Analyze module			
construct_feedback	*feedback_instructions*	*feedback*	InsEmptyError

Table 21: Feedback Access Routines

7.13.1 Uses

7.13.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: obtains a frame from Main module			
get_frame	*processed_frame*	NA	NoFrameError
Description: calculates metrics from frame provided by Main module			
calculate_frame	*processed_frame*	*angle* *height* *duration*	NoFrameError
Description: obtains metrics that need to be adjusted by the user (calls Analysis)			
fixed_frame	*NA*	*fixed_metrics*	NoFrameError

Table 22: Access Routines for Analysis Module

7.13.3 Semantics

State Variables

State Invariant

7.13.4 Assumptions

7.13.5 Access Routine Semantics

7.13.6 Local Functions

7.13.7 Local Functions

7.14 Module - Draw

NA

Access Routine Name	Input	Output	Exceptions
Description: measures the angle between the starting and final plane			
measure_angle	*initial_plane* *final_plane*	*angle*	FrameEmptyError
Description: measures difference in initial and final height			
measure_height	*initial_height* *final_height*	*height*	FrameEmptyError
Description: measures duration of the performed exercise			
measure_duration	*initial_time* *final_time*	*duration*	FrameEmptyError

Table 23: Metrics Access Routines

7.14.1 Uses

7.14.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
Description: Checks if the recording environment is appropriate (e.g., lighting, background)			
record_check	*starting_frane*	*good_to_record*	UserNotFound Error DownloadError
Description: Analyzes the user's starting form against a correct form model			
form_check	*processed_frames_list*	*feedback_instructions*	UserNotFound Error DownloadError
Description: Overlays pose estimation points onto the video feed for user feedback			
draw_points	*unprocessed_frames*	processed_frames_list	UserNotFound Error DownloadError

Table 24: Video Processing Access Routines

7.14.3 Semantics

State Variables
TODO
State Invariant
TODO

7.14.4 Assumptions

TODO

7.14.5 Access Routine Semantics

TODO

7.14.6 Local Functions

Access Routine Name	Input	Output	Exceptions
Description: Calls MediaPipe's draw functionality			
mpipe_draw	*unprocessed_frames*	*processed_frames_list*	MPbusyError

Table 25: Account Management Access Routines

7.15 Module - UI

NA

7.15.1 Uses

7.15.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
placeholder	*NA*	UserAccount_P OR UserAccount_PT	User_not_found Download_error

7.15.3 Semantics

TBD
State Variables

TODO
State Invariant
TODO

7.15.4 Assumptions

7.15.5 Access Routine Semantics

7.15.6 Local Functions

7.15.7 Local Functions

TODO

Access Routine Name	Input	Output	Exceptions
placeholder	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired

Table 27: UI Access Routines

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering.* Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach.* International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

8 Appendix

NA

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?
5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)
6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

Group - Reflection

1. N/A
2. N/A

3. N/A

4. I think that our other documents such as the requirements, hazard analysis need to be slightly adjusted as a result of the design documents. With the requirements specifically some of them pertain to how the application design is and after completing our initial draft of the MG and MIS, the design does not necessarily line up with the requirements document. Further, some of the hazards considered should be changed to include a couple more areas for hazards to occur. Lastly for both of the documents, the assumptions should be changed to incorporated design changes that we had not considered during that deliverable.

5. TODO

6. TODO

Matthew - Reflection

1. I think something that went well was our changes to overall workflow. One thing that we recognized was starting our document on google docs and then transferring our to sections to GitHub on the last two days the the deliverable was due led to some things going unchecked and the deliverable not turning out the way we had planned. Further it had also resulted in a 'panic' when we had to resolve merge conflicts between our sections soon to the deadline. Overall, we cut google docs for this deliverable and mainly focused on our sections within GitHub and doing multiple small PRs within the deliverable timeline and that had gone quite well. There are a few days left but a majority of the document has at least been accounted for and hopefully this workflow works for our team.

2. I think one pain point I experienced was time management. I had suggested that since the computing team's workload was quite different to the documentation team's, we could take on small sections within the document with hopes of submitting it early into the project. This did partially go well but with focus split in two areas it was hard to focus on the development side of things for the POC demo. This may be something that improves with time but as of now is currently a work in progress as development on my end is not where I want it to be.

3. I think that our design decisions partially came from speaking with our stakeholder (Kevin Yu) but other decisions were made with typical application creation. For example, the high-level was talked through with Kevin to simulate the timing and feedback delivery that you would encounter during physiotherapy but things like reporting, logging-in as well as account creation were from looking at other applications and aspects that went well from that end.

4. Group Q

5. Group Q

6. Group Q

Vaisnavi - Reflection

1. Something that went well was how we worked on creating more constant pull requests (PRs) to avoid merge conflicts. This worked out well with everyone consistently pushing more PRs early on, allowing someone on the team to review and effectively merge it in. Overall, the organization and collaboration went very smoothly, and it allowed us to ultimately finish ahead of our internal deadline so we had enough time for editing and formatting.

2. I think one of the major pain points through this deliverable was trying to balance both the development side for the POC demo while also trying to contribute to the document. As the main sub-team for the POC demo, we did take on smaller sections, but it was difficult to find that balance in time for both at the start.

3. We believe that most of our major design decisions stemmed directly from our meetings with our stakeholder, Kevin Yu. Throughout the development of this deliverable, a number of ideas that we originally planned had to be changed or completely re-worked based on the feedback we received. In several cases, Kevin helped us recognize that certain features were either not feasible within the course timeline or did not align with the true purpose and scope of our project. This was extremely helpful overall to have this input to tackle these decisions early on in the process.

4. Group Q

5. Group Q

6. Group Q

Maham - Reflection

1. Despite our (extremely) busy schedules, we made time to meet when we could and offer assistance to any member that needed it. In addition, all of us kept in contact and checked in with each other. I liked that none of us completely abandoned this project, but rather balanced it quite well. Also, the more frequent PR requests strategy (implemented for this deliverable) ended up being immensely helpful.

2. A pain point for me was that a lot of my time and energy was taken up by external tasks. These were, unfortunately, appointments I could not compromise on because they had been scheduled months in advance. I would have loved to give this project more of my focus, and I tried my best to self-organize, but there were some areas I fell short. I will attempt to refine my balancing act for the future.

3. The project is still in the early stages of development, and as a result, we do not have many concrete elements. However, some of developed ideas and design decisions was aided by Kevin, our supervisor. At this stage, we are constructing the foundation of the software, and that knowledge was supplied by him. Eventually, when we do have a functional product, we will want to loop in other stakeholders, e.g. patients, for modifications

4. Group Q

5. Group Q

6. Group Q

Cieran - Reflection

1. During this deliverable there was a need to agree upon implementation tactics. The team met and nailed down an approach for the development of the proof of concept, and I think everyone did a good job of ensuring their thoughts on what the system *was* got at least mentioned when developing the system.

2. I yet again sharked my teammates with procrastination. I have apologized profusely for my lack of initiative with this deliverable and have taken steps to ensure progress is far more consistent and meaningful in the future. As for this deliverable, I worked to catch up my own lack of effort towards the end of the it.

3. I think the design decisions, specifically around the system's allowance for physiotherapists to alter patient's exercise routines, were very influenced by meetings with our advisor. As a team, we wanted to ensure that the system was something wanted by both patients and physiotherapists; designing based off of requirements from past physiotherapy patients and our physiotherapist advisor was a must. Some design decisions, such as checking against public databases for the physiotherapist's identification number, were made in an attempt at security for users of the application.

4. GROUP Q

5. GROUP Q

6. GROUP Q